⚙ J.A.R.V.I.S PROJECT

# J.A.R.V.I.S.

Product Requirements Document (PRD)

| | |
|---|---|
| **Document Type** | **PRD** |
| **Version** | **v1.0** |
| **Author** | You (JARVIS Project) |
| **Date** | May 2025 |
| **Status** | DRAFT – In Review |

# 1. Executive Summary

J.A.R.V.I.S. (Just A Rather Very Intelligent System) is a locally-hosted, voice-activated AI personal assistant inspired by Iron Man's iconic AI. It runs entirely on the user's laptop without requiring any internet connection or cloud services. The assistant can hear voice commands, reason using a large language model, take actions on the computer (open apps, manage files, control the browser), write and execute code, and speak back in natural language.

# 2. Product Vision & Goals

## 2.1 Vision Statement

To build an offline, privacy-first AI assistant that feels like having a dedicated intelligent agent living inside your laptop — one that can operate your computer, write code, remember your context, and respond in a natural human voice.

## 2.2 Goals

- Operate 100% locally — no cloud API required for core functionality
- Respond to voice commands with sub-3-second latency on mid-range hardware
- Execute computer actions: open files, apps, websites, type text
- Generate, save, and run code in multiple languages
- Maintain memory of past sessions and user preferences
- Speak back in a clear, natural TTS voice
- Extendable via a plugin/tool system for future capabilities

# 3. Target User

## 3.1 Primary User

A single developer / power user who wants a personal productivity assistant to augment their workflow, automate repetitive tasks, and provide an AI coding companion — all without sacrificing privacy.

## 3.2 User Needs

| Pain Point | User Need | JARVIS Solution |
|---|---|---|
| Switching between apps manually | Launch apps & files by voice | System control via subprocess/PyAutoGUI |
| Repetitive coding boilerplate | Generate code on demand | LLM code generator tool |
| Searching and researching | Search & summarize the web | DuckDuckGo tool + LLM summary |

| | | |
|---|---|---|
| Privacy concerns with cloud AI | Everything runs offline | Ollama + local Whisper + local TTS |
| No persistent memory in AI tools | Remember context across sessions | ChromaDB vector memory |

# 4. Functional Requirements

## 4.1 Core Features (Must Have — MVP)

### FR-01: Voice Input

- System shall transcribe voice to text using OpenAI Whisper (local, whisper.cpp or Python)
- Shall detect a configurable wake word (default: 'Hey JARVIS')
- Shall accept fallback text input via terminal/GUI

### FR-02: LLM Brain

- System shall use Ollama with LLaMA 3.1 8B or Mistral 7B as the local LLM
- Shall maintain conversation history within a session
- Shall use a persistent JARVIS system prompt defining its persona and tool capabilities

### FR-03: Computer Control

- Shall open applications by name (chrome, vscode, notepad, etc.)
- Shall open files by path or name
- Shall open URLs in the default browser
- Shall simulate keyboard input and mouse actions via PyAutoGUI
- Shall run terminal/shell commands

### FR-04: Voice Output

- System shall convert LLM text responses to speech using pyttsx3 or Coqui TTS
- Shall support configurable voice, pitch, and speed

### FR-05: Code Generation

- Shall generate code in Python, JavaScript, HTML, bash, and others on request
- Shall save generated code to a specified file path
- Shall optionally execute Python/bash code and return output

## 4.2 Extended Features (Should Have — Phase 2)

### FR-06: Session Memory

- Shall store user interactions in ChromaDB (local vector database)
- Shall retrieve relevant past context to inject into LLM prompts

### FR-07: Web Search

- Shall search DuckDuckGo and return summarized results

- Shall optionally scrape and summarize a given URL

### FR-08: File Management

- Shall list, read, move, copy, rename, and delete files
- Shall summarize document contents (PDF, txt, docx)

### FR-09: GUI Interface

- Shall display a minimal desktop window with animated waveform during speech
- Shall show last command and JARVIS response as text

## 4.3 Future Features (Nice to Have — Phase 3)

- Calendar / reminder scheduling
- Email reading and composing
- Multi-model routing (fast vs deep queries)
- Vision capabilities (screenshot understanding)
- Home automation integration

# 5. Non-Functional Requirements

| Category | Requirement | Target Metric | Priority |
|---|---|---|---|
| Performance | Voice-to-response latency | < 3 seconds (CPU), < 1.5s (GPU) | **High** |
| Privacy | No data leaves device | Zero cloud calls for core features | **Critical** |
| Reliability | Uptime during session | > 99% (no crashes per 4hr session) | **High** |
| Usability | Wake word accuracy | > 90% correct detection | **High** |
| Portability | OS support | Windows 10/11, Ubuntu 20+, macOS 12+ | **Medium** |
| Extensibility | New tool integration | New tool addable in < 1 hour | **Medium** |

# 6. System Constraints

- Must run on minimum 8GB RAM laptop (16GB recommended)
- LLM inference must be possible on CPU (GPU acceleration is a bonus)
- No paid APIs required for core functionality
- All storage must remain local (no cloud databases)

- Must be launchable from a single command or startup shortcut

## 7. Success Metrics

| Metric | Definition | Target |
|---|---|---|
| Task Completion Rate | % of voice commands correctly executed | > 85% on standard command set |
| Voice Recognition Accuracy | Whisper transcription WER | < 10% Word Error Rate |
| Response Latency | Time from end of speech to spoken reply | < 3s on CPU, < 1.5s on GPU |
| Tool Coverage | # of tool types working at launch | > 10 distinct tools in MVP |
| Memory Relevance | Correct past context retrieved | > 80% relevance score on test set |

## 8. Out of Scope

- Mobile (Android/iOS) version
- Multi-user support
- Cloud deployment / server hosting
- Commercial distribution or SaaS model
- Integration with paid 3rd-party APIs (Slack, Notion, etc.) in MVP

⚙ J.A.R.V.I.S PROJECT

# J.A.R.V.I.S.

Technical Requirements Document (TRD)

| | |
|---|---|
| **Document Type** | **TRD** |
| **Version** | **v1.0** |
| **Author** | You (JARVIS Project) |
| **Date** | May 2025 |
| **Status** | DRAFT – In Review |

# 1. System Architecture Overview

JARVIS follows a modular pipeline architecture. Every voice command travels through six distinct layers: input capture, transcription, reasoning, tool dispatch, execution, and response. Each layer is independently swappable, ensuring the system remains upgradeable as better local models emerge.

## 1.1 High-Level Architecture

| Layer | Component | Technology | I/O |
|---|---|---|---|
| **L1** | Audio Capture | PyAudio + Porcupine | Mic → PCM audio stream |
| **L2** | Speech-to-Text | Whisper (faster-whisper) | PCM audio → text string |
| **L3** | LLM Reasoning | Ollama + LLaMA 3.1 8B | Text + tools → tool_call or text |
| **L4** | Tool Router | LangChain AgentExecutor | tool_call → function dispatch |
| **L5** | Tool Executors | Python stdlib + PyAutoGUI + Selenium | function call → OS action + result |
| **L6** | Text-to-Speech | pyttsx3 / Coqui TTS | Text → spoken audio |

# 2. Technology Stack (Detailed)

## 2.1 Local LLM — Ollama

- Installation: https://ollama.com → ollama pull llama3.1
- API: REST endpoint at http://localhost:11434/api/chat
- Recommended models: llama3.1:8b (8GB RAM), mistral:7b (6GB RAM), codellama:7b (for code tasks)
- Context window: 8192 tokens (configurable via Modelfile)
- Function calling: Use system prompt with JSON tool schema for structured output

## 2.2 Speech Recognition — faster-whisper

- Library: faster-whisper (CTranslate2-based, 4x faster than original)
- Model: whisper-base (fast, ~1GB) or whisper-small (more accurate, ~2GB)
- Language: English (set language='en' for 2x speed boost)
- VAD: Use Silero VAD for voice activity detection to avoid processing silence
- Input: 16kHz mono PCM via sounddevice or PyAudio

## 2.3 Wake Word — Porcupine

- Library: pvporcupine (free tier: 1 custom wake word)

- Custom wake word: Train 'Hey JARVIS' at console.picovoice.ai (free account)
- Fallback: Use a keyword list approach with webrtcvad if Porcupine license is restrictive

### 2.4 Text-to-Speech

- Primary: pyttsx3 (zero-dependency, offline, uses system TTS voices)
- High quality: Coqui TTS with XTTS-v2 model (cloneable voice, 500MB model)
- Ultra-low latency: kokoro-onnx (fastest local TTS, ~50ms)

### 2.5 Agent Framework — LangChain

- Use LangChain's Tool class to register each capability as a structured tool
- Use ChatOllama as the LLM backend
- Use AgentType.OPENAI_FUNCTIONS or ReAct agent depending on model capability
- Each tool has: name, description, args_schema (Pydantic), and _run() method

### 2.6 Memory — ChromaDB

- Persistent local vector database stored in ~/.jarvis/memory/
- Embedding model: sentence-transformers/all-MiniLM-L6-v2 (runs locally, 80MB)
- Each turn stored as: {role, content, timestamp, embedding}
- Retrieval: Top-3 most semantically similar past turns injected into prompt context

## 3. Module Specifications

### 3.1 Tool Registry — Core Tools

| Tool Name | Trigger Example | Implementation | Returns |
|---|---|---|---|
| open_app | "Open Chrome" | subprocess.Popen(['chrome']) | Success/error string |
| open_file | "Open my resume.pdf" | os.startfile() / xdg-open | Success/error string |
| open_url | "Go to GitHub" | webbrowser.open(url) | URL opened confirmation |
| type_text | "Type hello world" | pyautogui.typewrite(text) | Typed text confirmation |
| run_shell | "Run my build script" | subprocess.run(cmd, shell=True) | stdout/stderr output |
| write_code | "Write a Python web scraper" | LLM → write to file | File path + code preview |
| run_code | "Run the script" | subprocess.run(['python', path]) | Code execution output |
| search_web | "Search LangChain docs" | DuckDuckGo API + LLM summary | Summarized search result |
| read_file | "Read my notes.txt" | open(path).read() | File content string |

| | | | |
|---|---|---|---|
| list_files | "What is in my Downloads?" | os.listdir(path) | File listing string |
| take_screenshot | "Take a screenshot" | pyautogui.screenshot() | Image saved to disk |

## 3.2 Directory Structure

**jarvis/**

- main.py ← Entry point, starts all threads
- config.yaml ← All settings (model, voice, paths, wake word)
- core/
  - listener.py ← PyAudio + Porcupine wake word detection
  - transcriber.py ← faster-whisper STT
  - brain.py ← Ollama LLM interface + conversation history
  - agent.py ← LangChain agent + tool dispatch
  - speaker.py ← TTS engine abstraction
  - memory.py ← ChromaDB read/write helpers
- tools/
  - system.py ← open_app, open_file, run_shell, type_text
  - browser.py ← open_url, selenium actions
  - coder.py ← write_code, run_code
  - search.py ← search_web, summarize_url
  - files.py ← read_file, list_files, write_file
- ui/
  - window.py ← PyQt5 minimal HUD (optional)
- data/
  - memory/ ← ChromaDB persistent storage
  - generated_code/ ← LLM-generated scripts saved here

## 3.3 Data Flow — Sequence

1. User says: 'Hey JARVIS, open Chrome and search for Python tutorials'
2. Porcupine detects wake word → triggers recording
3. PyAudio records 5 seconds of audio
4. faster-whisper transcribes → 'open Chrome and search for Python tutorials'
5. Memory module retrieves top-3 relevant past interactions from ChromaDB
6. Brain builds prompt: [system_prompt + memory_context + user_message]
7. Ollama LLM returns tool_calls: [open_app('chrome'), search_web('Python tutorials')]
8. AgentExecutor dispatches to system.open_app('chrome') → opens Chrome
9. AgentExecutor dispatches to search.search_web('Python tutorials') → returns results

10. Brain generates final response: 'Chrome is open and here are the top Python tutorial sites...'
11. Speaker converts text to audio and plays it
12. Conversation saved to ChromaDB memory

# 4. API & Interface Contracts

## 4.1 LLM System Prompt Template

You are J.A.R.V.I.S., a highly intelligent and efficient AI personal assistant running locally on the user's laptop. You are witty, concise, and always address the user as 'Sir' or by their preferred name. You have access to the following tools: [TOOL_LIST]. Always use tools for computer actions. Respond concisely in 1-3 sentences when no tool is needed. Current time: [TIMESTAMP]. Recent memory context: [MEMORY_CONTEXT].

## 4.2 Tool Schema (LangChain)

Each tool is defined as a Python class inheriting from LangChain BaseTool with: name (str), description (str, used by LLM to decide when to call it), args_schema (Pydantic BaseModel defining parameters), and a _run(self, **kwargs) method returning a string result.

# 5. Infrastructure Requirements

## 5.1 Hardware Minimum Spec

| Component | Minimum | Recommended | Notes |
|---|---|---|---|
| RAM | 8 GB | 16 GB+ | 8B model needs ~6GB RAM |
| CPU | 4 cores, 2.5GHz | 8 cores, 3.5GHz+ | More cores = faster inference |
| GPU (Optional) | None | NVIDIA 4GB VRAM+ | 10-20x faster with GPU |
| Storage | 15 GB free | 30 GB+ SSD | Models take 4-8GB each |
| Microphone | Any built-in mic | USB mic or headset | Better mic = better accuracy |

## 5.2 Software Dependencies

| Package | Version | Purpose | Install |
|---|---|---|---|
| Ollama | latest | Local LLM server | ollama.com installer |
| faster-whisper | ^1.0 | Speech-to-text | pip install faster-whisper |

| | | | |
|---|---|---|---|
| langchain | ^0.3 | Agent + tool framework | pip install langchain |
| langchain-ollama | latest | Ollama LangChain binding | pip install langchain-ollama |
| chromadb | ^0.5 | Vector memory DB | pip install chromadb |
| sentence-transformers | ^3.0 | Local embeddings | pip install sentence-transformers |
| pyautogui | latest | Mouse/keyboard control | pip install pyautogui |
| pyttsx3 | ^2.9 | Text-to-speech | pip install pyttsx3 |
| pvporcupine | ^3.0 | Wake word detection | pip install pvporcupine |
| sounddevice | latest | Audio recording | pip install sounddevice |
| selenium | ^4.0 | Browser automation | pip install selenium |
| PyQt5 | ^5.15 | Desktop GUI (optional) | pip install PyQt5 |

# 6. Security & Privacy

- All LLM inference stays local — no data sent to external servers
- Shell command execution must require explicit user confirmation for destructive operations (delete, format)
- A config.yaml whitelist of allowed shell commands should be enforced
- Porcupine wake word detection is on-device with no audio upload
- ChromaDB memory stored in user's home directory with standard OS file permissions
- No API keys required; if added (for future integrations), keys stored in .env, never committed to git

# 7. Error Handling Strategy

| Error Type | Detection | Fallback | User Message |
|---|---|---|---|
| LLM offline | HTTP connection refused | Retry 3x, then fail | "JARVIS brain offline, Sir" |
| STT failure | Whisper returns empty | Re-prompt for input | "Could not hear you, please repeat" |
| Tool execution error | Exception in _run() | Return error string to LLM | "Could not complete that task" |
| App not found | FileNotFoundError | Search PATH, suggest alternatives | "App not found on your system" |
| TTS failure | pyttsx3 exception | Print to console instead | Text displayed on screen |

⚙ J.A.R.V.I.S PROJECT

# J.A.R.V.I.S.

Detailed Project Roadmap

| | |
|---|---|
| **Document Type** | **ROADMAP** |
| **Version** | **v1.0** |
| **Author** | You (JARVIS Project) |
| **Date** | May 2025 |
| **Status** | DRAFT – In Review |

# 1. Roadmap Overview

The JARVIS project is structured into 5 phases spanning approximately 10-14 weeks for a solo developer working part-time (2-4 hours/day). Each phase builds on the last, ensuring you always have a working (if limited) system from Week 1 onward. The philosophy is: get it running first, then make it smarter.

| Phase | Name | Duration | Deliverable | Status |
|---|---|---|---|---|
| **Phase 1** | The Brain | Week 1-2 | LLM working via text | Not Started |
| **Phase 2** | The Hands | Week 3-4 | Computer control tools | Not Started |
| **Phase 3** | The Voice | Week 5-6 | Full voice I/O + wake word | Not Started |
| **Phase 4** | The Memory | Week 7-8 | Persistent memory + web search | Not Started |
| **Phase 5** | The Suit | Week 9-12 | GUI, polish, full integration | Not Started |

# 2. Phase 1 — The Brain (Weeks 1-2)

Goal: Get Ollama running locally and build a text-based chat loop with the JARVIS persona. By the end of this phase, you should be able to type commands and get intelligent responses in the terminal.

## 2.1 Setup Tasks

1. Install Python 3.11+ from python.org
2. Install Ollama from ollama.com
3. Pull the LLaMA 3.1 model: ollama pull llama3.1
4. Create project folder: mkdir jarvis && cd jarvis
5. Create virtual environment: python -m venv venv && venv\Scripts\activate
6. Install LangChain: pip install langchain langchain-ollama

## 2.2 Development Tasks

7. Write brain.py — Ollama chat wrapper with conversation history
8. Define the JARVIS system prompt with persona and tool descriptions
9. Write a CLI loop: user types → LLM responds → loop
10. Test with 20+ sample queries to validate response quality
11. Tune system prompt until responses feel consistently JARVIS-like

### 2.3 Acceptance Criteria

- Can hold a 10-turn conversation without losing context
- Responds in JARVIS style (concise, intelligent, slightly witty)
- Response time < 10 seconds on CPU (acceptable for now)
- Handles 'I don't know' gracefully without hallucinating

## 3. Phase 2 — The Hands (Weeks 3-4)

Goal: Give JARVIS the ability to control your computer. By the end of this phase, you can type a command like 'open Chrome' and JARVIS will actually do it — not just describe it.

### 3.1 Setup Tasks

12. Install tools: pip install pyautogui selenium webdriver-manager
13. Download ChromeDriver for Selenium (or use webdriver-manager auto-download)
14. Create tools/ directory structure

### 3.2 Development Tasks

15. Implement system.py: open_app, open_file, run_shell, type_text, take_screenshot
16. Implement browser.py: open_url, selenium-based web interaction
17. Implement coder.py: write_code (LLM generates → saves to file), run_code
18. Implement files.py: read_file, write_file, list_files, delete_file
19. Build agent.py using LangChain AgentExecutor with all tools registered
20. Wire tools into brain.py — LLM now decides which tool to call
21. Add basic error handling: each tool returns error string on failure

### 3.3 Milestone Test — The 10 Command Challenge

**Test these 10 commands — all must work before moving to Phase 3:**

- 'Open Notepad'
- 'Open Google Chrome'
- 'Go to github.com'
- 'Type Hello World'
- 'List files in my Downloads folder'
- 'Write a Python hello world script and save it'
- 'Run the script you just wrote'
- 'Take a screenshot'
- 'Read the file notes.txt'
- 'Search the web for Python tutorials'

# 4. Phase 3 — The Voice (Weeks 5-6)

Goal: Replace typing with speaking and listening. By the end of this phase, JARVIS will hear you say 'Hey JARVIS' and respond with a spoken voice — just like the movies.

## 4.1 Setup Tasks

22. Install audio packages: pip install faster-whisper sounddevice pvporcupine pyttsx3
23. Create a free Picovoice account and train 'Hey JARVIS' wake word
24. Download wake word .ppn file from Picovoice console
25. Test microphone works: python -c 'import sounddevice; print(sounddevice.query_devices())'

## 4.2 Development Tasks

26. Build listener.py: Porcupine detects wake word → triggers recording
27. Build transcriber.py: sounddevice captures audio → faster-whisper transcribes
28. Build speaker.py: pyttsx3 converts text to speech with JARVIS voice settings
29. Update main.py: run listener in background thread, pipe audio through pipeline
30. Add voice confirmation phrases: 'On it, Sir', 'Done', 'Opening now'
31. Handle ambient noise: add minimum confidence threshold for transcription

## 4.3 Voice Pipeline Optimization

- Use Whisper base.en model (English-only, fastest)
- Set VAD filter=True in faster-whisper to skip silence
- Run TTS in a separate thread so JARVIS can speak while next command is being processed
- Add a 'Stop' or 'Cancel' keyword to interrupt current action

## 4.4 Acceptance Criteria

- Wake word detection works from 1.5m distance in quiet room
- Full voice loop latency (speak → JARVIS speaks back) < 4 seconds on CPU
- Transcription accuracy > 85% for clear speech
- Can have 5-turn voice conversation without touching keyboard

# 5. Phase 4 — The Memory (Weeks 7-8)

Goal: Give JARVIS long-term memory so it remembers past conversations, your preferences, and frequently used commands across sessions.

## 5.1 Setup Tasks

32. Install: pip install chromadb sentence-transformers duckduckgo-search
33. Verify sentence-transformers model downloads on first run
34. Create ~/.jarvis/memory/ directory for persistent storage

### 5.2 Development Tasks

35. Build memory.py: ChromaDB client + add_memory() + retrieve_context() functions
36. Integrate memory into brain.py: retrieve top-3 related memories before each LLM call
37. Save each completed turn to memory with timestamp and metadata
38. Implement search.py: DuckDuckGo search + LLM-based summarization
39. Add user preferences memory: JARVIS remembers your name, preferred browser, project paths
40. Build a 'forget' command: 'JARVIS, forget that'

### 5.3 Advanced Memory Features

- Daily summary: JARVIS summarizes today's interactions at the end of each session
- Proactive recall: If you mention a project, JARVIS retrieves related past context automatically
- Memory browser: A simple CLI command to view/search stored memories

## 6. Phase 5 — The Suit (Weeks 9-12)

Goal: Polish, UI, and full integration. Make JARVIS feel like a real product — not just a collection of scripts.

### 6.1 GUI Development (Weeks 9-10)

41. Install: pip install PyQt5
42. Build minimal HUD window: waveform animation when listening, response text display
43. Add system tray icon so JARVIS runs in background
44. Create startup script that launches JARVIS on login
45. Add a settings panel: change wake word, voice speed, model choice

### 6.2 Polish & Personality (Week 10-11)

46. Fine-tune system prompt for consistent JARVIS personality
47. Add Easter eggs: JARVIS responds to 'Are you Skynet?' and similar references
48. Add time-aware responses: 'Good morning, Sir' vs 'Good evening, Sir'
49. Add a help command: 'What can you do?' → JARVIS lists all capabilities
50. Record 5 random opening phrases for variety

### 6.3 Reliability & Testing (Week 11-12)

51. Write unit tests for every tool using pytest
52. Write integration tests for the full voice pipeline
53. Stress test: run 100 consecutive commands, measure success rate
54. Fix top 10 failure modes identified in testing
55. Write a setup.sh / setup.bat installer script for easy reinstall

### 6.4 Stretch Goals (Post Week 12)

- Vision tool: screenshot → ask LLM 'what's on screen?'
- Email integration via Gmail API
- Reminder/scheduler: 'Remind me in 30 minutes'
- Multi-model routing: fast model for simple queries, powerful model for complex ones
- Custom Coqui TTS voice trained on a voice sample of your choice

## 7. Sprint Planning (Detailed Week-by-Week)

| Week | Phase | Tasks | Deliverable |
|---|---|---|---|
| **W1** | Phase 1 | Install Python, Ollama, pull model, basic chat loop | CLI JARVIS chat working |
| **W2** | Phase 1 | Tune system prompt, add conversation history, test 20+ queries | Stable JARVIS persona |
| **W3** | Phase 2 | Build system.py and files.py tools, wire to LangChain agent | Open apps + files working |
| **W4** | Phase 2 | Build browser.py + coder.py, run 10-command challenge | All core tools operational |
| **W5** | Phase 3 | Integrate faster-whisper STT, test transcription accuracy | Voice input working |
| **W6** | Phase 3 | Add Porcupine wake word + pyttsx3 TTS, full voice loop | 'Hey JARVIS' working end-to-end |
| **W7** | Phase 4 | Build ChromaDB memory, integrate into conversation pipeline | JARVIS remembers past sessions |
| **W8** | Phase 4 | Add web search tool, user preferences memory, forget command | Search + memory complete |
| **W9** | Phase 5 | Build PyQt5 HUD window with waveform animation | Visual JARVIS HUD |
| **W10** | Phase 5 | System tray icon, startup launch, settings panel | JARVIS runs on startup |
| **W11** | Phase 5 | Personality polish, Easter eggs, help command | Full personality integration |
| **W12** | Phase 5 | Unit tests, stress testing, 100-command test, installer script | Stable production-ready JARVIS |

## 8. Risk Register

| Risk | Likelihood | Impact | Mitigation |
|---|---|---|---|
| LLM too slow on CPU | **High** | **High** | Use quantized GGUF models (Q4_K_M); consider GPU upgrade or cloud fallback |
| Wake word false positives | Medium | Low | Add 1-second debounce; retrain wake word with more samples |
| Tool execution causes system damage | Low | **High** | Implement command whitelist; require confirmation for destructive actions |
| Whisper accuracy poor in noisy environment | Medium | Medium | Use directional USB mic; implement noise gate; add text fallback input |
| LangChain version incompatibility | Medium | Medium | Pin all dependencies in requirements.txt; test on clean venv regularly |
| Ollama model not following tool schema | Medium | **High** | Use structured output prompting; switch to mistral or codellama for tool calls if needed |

# 9. Definition of Done

**JARVIS is considered complete (v1.0) when all of the following are true:**

- Wake word reliably triggers listening from normal conversational distance
- All 10 Phase 2 commands work 90%+ of the time
- Voice loop latency is under 5 seconds on the target hardware
- JARVIS remembers at least 3 facts from previous sessions correctly
- GUI HUD runs without crashing for 4+ hours continuously
- Installer script sets up a fresh environment in under 15 minutes
- JARVIS has a unique personality that feels consistently 'JARVIS-like'

# 10. Resources & References

- Ollama: https://ollama.com — Local LLM server
- faster-whisper: https://github.com/SYSTRAN/faster-whisper
- LangChain Docs: https://python.langchain.com/docs
- ChromaDB: https://docs.trychroma.com
- Porcupine Wake Word: https://picovoice.ai/platform/porcupine
- PyAutoGUI: https://pyautogui.readthedocs.io
- Coqui TTS: https://github.com/coqui-ai/TTS
- sentence-transformers: https://sbert.net